

SYSTEM ADMIN WITH IT SERVICE MANAGEMENT & AI EMPHASIS

YEAR : 3rd YEAR

COLLEGE CODE : MSU220

COLLEGE NAME : Sri Sankara Bhagavathi Arts and Science
College,Kommadikottai.

DEPARTMENT : BSC IT

SEMESTER : 5

TITLE : Implement Client Script & UI Policy (Incident)

GOOGLE DRIVE LINK:

https://drive.google.com/file/d/1n-FMtacxT7LORkD4AD_BSkFDU2I699Kq/view?usp=sharing

GITHUB LINK :

<https://github.com/subithra2027-lgtm/Implement-Client-Script-UI-Policy-Incident->

TEAM DETAILS :

Team Member 1

Name : A.Subithra

Register No: 24082201808412109

NM ID : f592de07fd4a9db9e92459c2de5f4a84

MAIL ID : subithra2027@gmail.com

Team Member 2

Name : N.Mahanattar

Register No: 24082201808412105

NM ID : FF2C99235B7146E8CDF4BCFA62A55F62

MAIL ID : mahanattar464@gmail.com

Team Member 3

Name : K.Balakrishnan
Register No: 24082201808411102
NM ID : F47F31DE271354BBC5B4B00FC0DA211F
MAIL ID : kbalakrishnan022@gmail.com

Team Member 4

Name : G.Rajasekar
Register No: 24082201808411108
NM ID : 8ACC7B5E07FF24A897B4640EA526BD78
MAIL ID : rajassekar007@gmail.com

Team Member 5

Name : N.Magesh kumar
Register No: 24082201808411104
NM ID : BC15DE39579F5129E1542F0C5EDE0FE4
MAIL ID : mageshking76@gmail.com

Team Size: 5 Members

ABSTRACT

The project “**Implement Client Script & UI Policy (Incident)**” focuses on improving data accuracy in ServiceNow Incident Management. UI Policies and Client Scripts are used to control field behavior, automatically update values, validate user inputs, and prevent incorrect Incident updates. The project is tested through different Incident scenarios to ensure proper functionality. Overall, it helps reduce data entry errors and maintain consistent Incident records.

1. INTRODUCTION

ServiceNow Incident Management helps organizations manage and resolve incidents efficiently. However, incorrect or incomplete data entry can affect the incident handling process.

This project implements **UI Policies and Client Scripts** on the Incident table to control field behavior, automatically update values, validate user inputs, and

prevent incorrect updates. The project demonstrates how these features can improve data accuracy and provide a consistent Incident Management process.

2. BRAINSTORMING & IDEATION PHASE

2.1 Problem Identification

In ServiceNow Incident Management, users may enter incomplete, inconsistent, or incorrect information while creating or updating Incident records. Manual checking of every field can also lead to errors and affect the incident management process. Therefore, a controlled method is required to validate information and dynamically manage Incident fields based on specific conditions.

2.2 Project Idea

The project idea is to implement **UI Policies and Client Scripts** in the ServiceNow Incident table. These features are used to dynamically control fields, automatically set values, validate user input, and restrict unwanted changes.

2.3 Proposed Solution

The proposed solution is to configure the Incident form with appropriate UI Policies and Client Scripts. When an Incident has **High Impact**, the system applies the required field controls and validations automatically. This helps users enter correct information and reduces errors during Incident creation and updating.

2.4 Objective

The main objectives of the project are:

- ❖ To improve data accuracy in Incident records.
- ❖ To control field behavior dynamically.
- ❖ To automatically update required values.
- ❖ To validate Incident records before submission.
- ❖ To prevent unwanted changes through list editing.

3. REQUIREMENT ANALYSIS PHASE

3.1 Functional Requirements

The project should provide the following functionalities:

- Create a UI Policy for High Impact Incidents.
- Make Assignment Group mandatory when Impact is High.
- Make the Urgency field read-only when Impact is High.
- Automatically set Urgency to High when Impact is High.
- Prevent saving when Assigned To is empty for High Impact incidents.
- Prevent State changes through direct list editing.
- Allow State changes through the Incident form.
- Reverse the UI Policy behavior when Impact is changed from High to another value.

3.2 Software Requirements

The following software and platform are required:

- **ServiceNow Personal Developer Instance (PDI)**
- **Web Browser**
- **ServiceNow Incident Management**
- **UI Policy**
- **UI Policy Actions**
- **Client Scripts**

3.3 Skills Required

The project requires basic knowledge of:

- ServiceNow Incident Management
- UI Policies
- UI Policy Actions
- Client Scripts
- Basic JavaScript
- Form Validation

4. PROJECT DESIGN PHASE

4.1 System Design

The project is designed to control the behavior of Incident records based on the values entered by the user. **UI Policies and Client Scripts** work together to provide dynamic field control and validation.

The basic workflow is:

Incident Form → Impact = High → UI Policy / Client Script → Field Behavior & Validation → Record Submission

4.2 UI Policy Design

A UI Policy named **High Impact Control** is created for the Incident table.

The condition is set to **Impact is 1 – High**. When this condition is satisfied, the Assignment Group field becomes mandatory. The **Reverse if false** option is enabled to restore the normal behavior when the condition is no longer true.

4.3 UI Policy Action Design

A UI Policy Action is configured for the **Urgency** field. When the Impact is High, the Urgency field becomes **read-only**.

4.4 Client Script Design

Three Client Scripts are designed for different purposes:

onChange: Automatically sets Urgency to High when Impact is High.

onSubmit: Prevents saving when Assigned To is empty for a High Impact Incident.

onCellEdit: Prevents State changes through direct list editing.

4.5 Overall Workflow

Impact = High



Assignment Group → Mandatory



Urgency → Read-only & Automatically set to High



Assigned To → Required before saving



State → Cannot be changed through list editing

5. PROJECT PLANNING PHASE

5.1 Development Plan

The project was planned in a step-by-step manner to ensure that all required UI Policies and Client Scripts were implemented and tested properly.

The development process was divided into five main tasks followed by functional testing.

5.2 Task Planning

Task 1 – UI Policy Creation

Create the **High Impact Control** UI Policy for the Incident table and configure the High Impact condition.

Task 2 – UI Policy Action

Configure the **Urgency** UI Policy Action to make the field read-only when Impact is High.

Task 3 – onChange Client Script

Create a Client Script to automatically set Urgency to High when Impact changes to High.

Task 4 – onSubmit Client Script

Create a Client Script to prevent submission when Assigned To is empty for a High Impact Incident.

Task 5 – onCellEdit Client Script

Create a Client Script to prevent direct State changes through Incident list editing.

5.3 Testing Plan

After completing the development tasks, the project was tested using different Incident scenarios.

The testing includes:

- High Impact Incident validation.
- Successful Incident submission.
- Reverse condition testing.
- State list-edit restriction.

- State form-based update.

5.4 Project Execution Flow

Planning



UI Policy Configuration



Client Script Configuration



Functional Testing



Result Verification



Final Demonstration

5.5 Expected Outcome

The planned implementation is expected to provide proper field validation, automatic value updates, controlled field behavior, and prevention of unwanted Incident modifications.

6. PROJECT DEVELOPMENT PHASE

6.1 Task 1 – Create UI Policy

A UI Policy named **High Impact Control** was created for the **Incident** table. The condition was configured as **Impact is 1 – High**. The policy was activated and the **Reverse if false** option was enabled. The Assignment Group field was configured as mandatory for High Impact incidents.

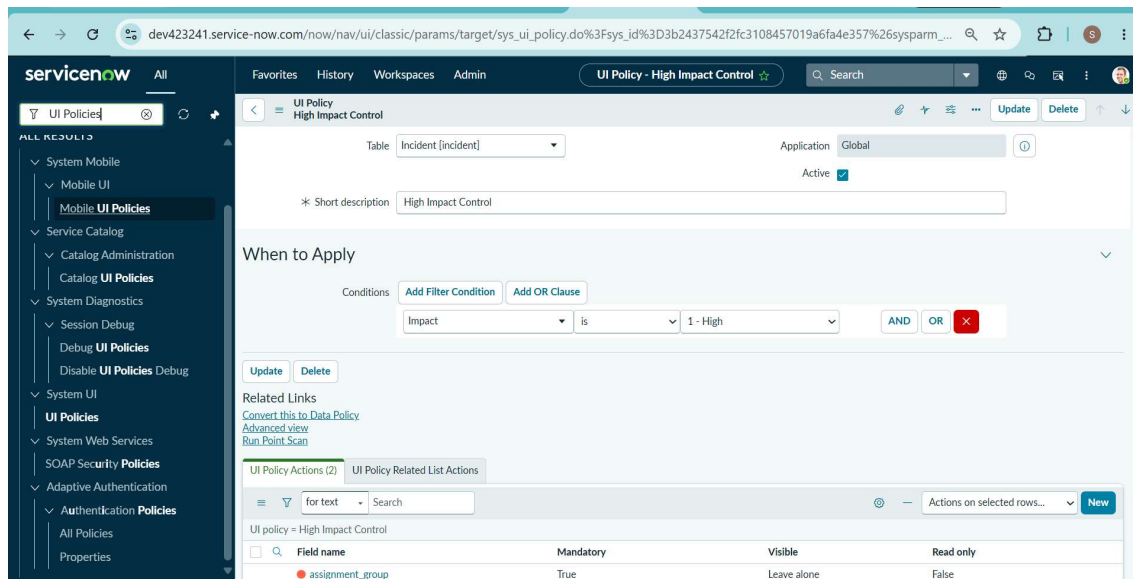


Figure 1: High Impact Control UI Policy

6.2 Task 2 – Configure UI Policy Action

A UI Policy Action was created under the **High Impact Control** policy for the **Urgency** field.

The **Read-only** option was enabled. Therefore, when the Impact is High, the Urgency field cannot be manually modified.

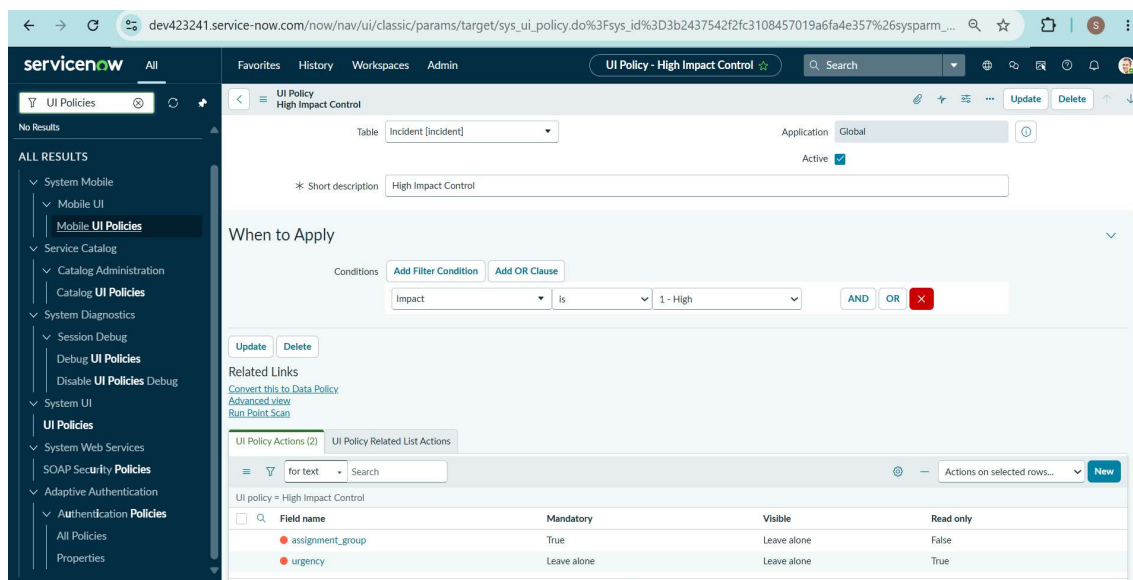


Figure 2: Urgency UI Policy Action

6.3 Task 3 – Create onChange Client Script

An **onChange Client Script** named **Auto set urgency for high impact** was created for the Incident table.

The script is triggered when the **Impact** field is changed. When the Impact is set to High, the system automatically sets the Urgency to High and displays an information message.

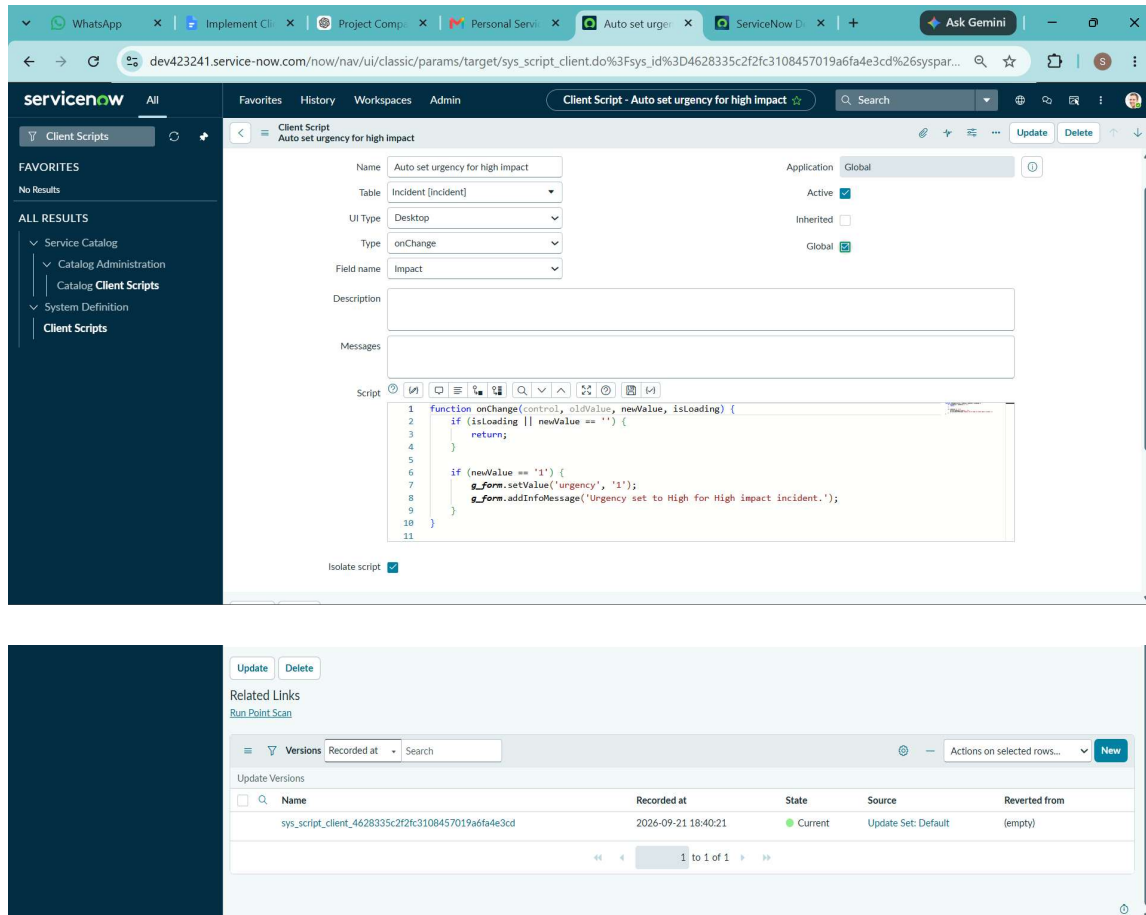


Figure 3: Auto Set Urgency Client Script

Task 3 – Working Result

When Impact was changed to High, the Urgency field was automatically updated to High.

Figure 4: Automatic Urgency Update

6.4 Task 4 – Create onSubmit Client Script

An **onSubmit Client Script** named **Prevent save if Assigned To missing** was created for the Incident table. The script checks whether the Impact is High and whether the Assigned To field is empty. If both conditions are satisfied, the system prevents the Incident from being saved and displays an error message.

Update Versions				
Name	Recorded at	State	Source	Reverted from
sys_script_client_403a71102f6c3108457019a6fa4e3d1	2026-09-21 18:46:05	Current	Update Set: Default	(empty)

Figure 5: Prevent Save Client Script

Task 4 – Working Result

When Impact was High and Assigned To was empty, the system prevented the Incident from being submitted.

dev423241.service-now.com/now/nav/ui/classic/params/target/incident.do%3Fsys_id%3D-1%26sysparm_clear_stack%3Dtrue

Incident - Create INC0010006

Urgency set to High for High Impact Incident.

The following mandatory fields are not filled in: Short description

Number: INC0010006

* Caller: Abel Tuter

Category: Inquiry / Help

Subcategory: -- None --

Service:

Service offering:

Configuration item:

Channel: -- None --

State: New

Impact: 1 - High

Urgency: 1 - High

Priority: 1 - Critical

* Assignment group: CAB Approval

Assigned to:

* Short description: hi

Description:

Assigned To is mandatory for High Impact Incidents.

Assigned To is mandatory for High Impact Incidents.

Figure 6: Assigned To Validation

6.5 Task 5 – Create onCellEdit Client Script

An **onCellEdit Client Script** named **Prevent state change via list edit** was created for the **State** field.

The script prevents users from directly changing the Incident State through list editing and displays an alert instructing the user to open the Incident form.

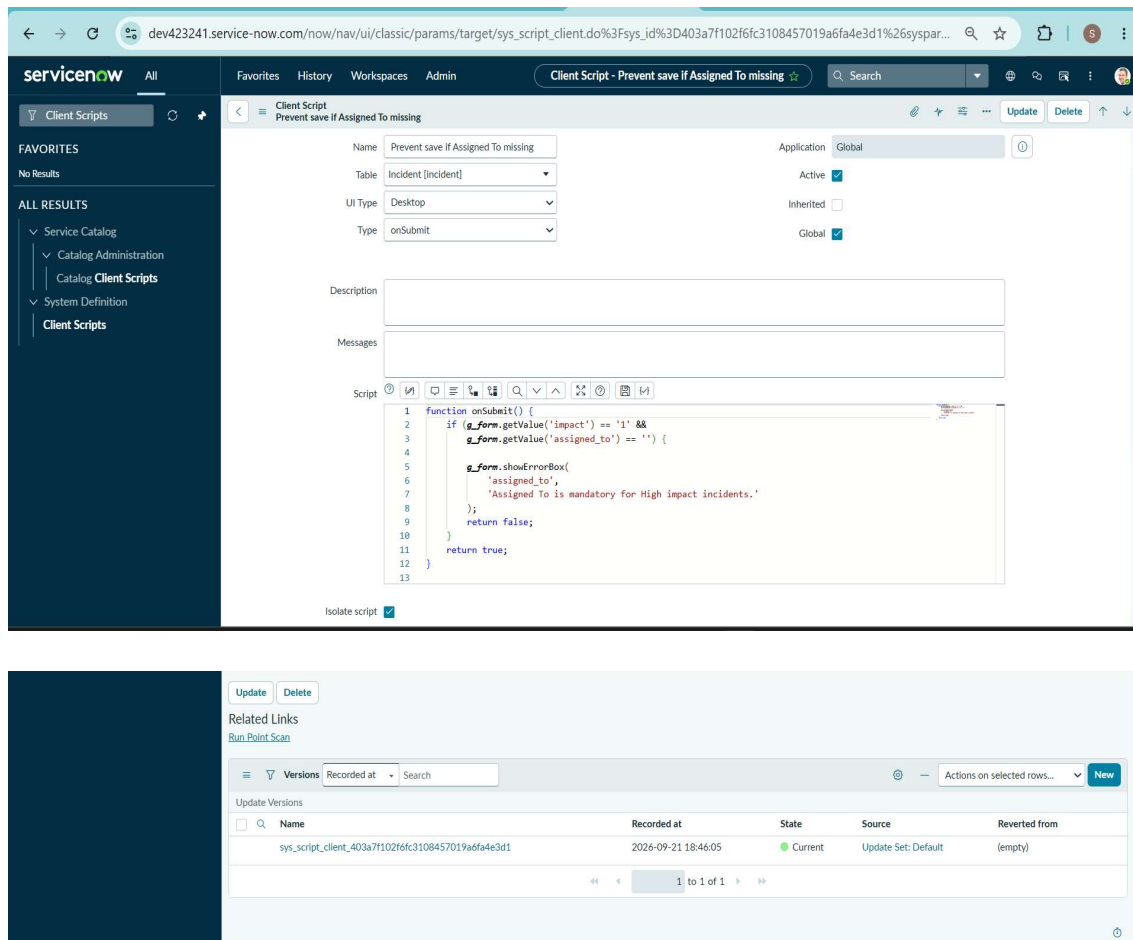


Figure 7: Prevent State Change Client Script

Task 5 – Working Result

When the State was edited directly from the Incident list, an alert message was displayed and the State change was blocked.

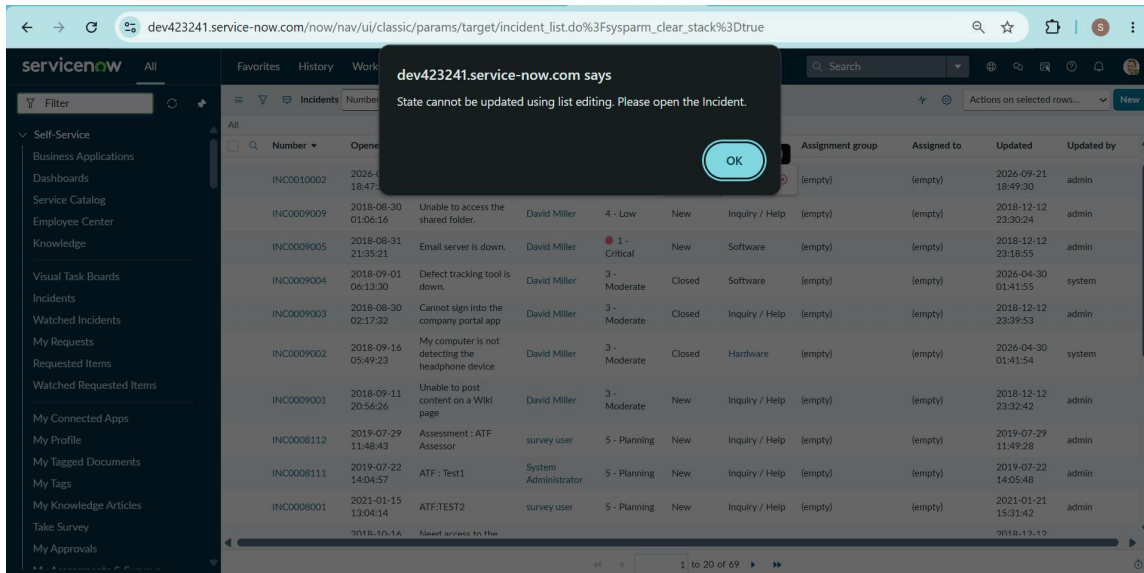


Figure 8: State List Edit Restriction

7. PROJECT TESTING PHASE

7.1 Activity 1 – Create New Incident

A new Incident was created and the **Impact** was set to **1 – High** while **Assigned To** was left empty. When Submit was selected, the system prevented the Incident from being saved and displayed the validation message.

Expected Result:

The Incident should not be saved when Assigned To is empty for a High Impact Incident. The UI Policy and Urgency behavior should also work correctly.

Actual Result:

The validation worked successfully and the Incident was not submitted without the required information.

Screenshot:

7.2 Activity 2 – Successful Incident Submission

The **Assigned To** field was filled and the Incident was submitted again. The Incident was saved successfully. The automatic Urgency setting and other field restrictions were also verified.

Expected Result:

The Incident should be saved successfully after providing the required information.

Actual Result:

The Incident was saved successfully and the configured rules worked as expected.

Screenshot:

7.3 Activity 3 – Reverse Condition Test

An existing Incident **INC0010003** was opened with Impact set to **High**. The Impact was changed from **High** to **Medium**.

Expected Result:

When Impact is changed from High to another value, **Assigned To** should no longer be mandatory and **Urgency** should become editable. The Incident should be saved successfully.

Actual Result:

The reverse condition worked successfully. Assigned To was no longer mandatory, Urgency became editable, and the Incident was saved.

Screenshot:

The screenshot shows the ServiceNow interface for incident INC0010003. The form is titled 'Incident - INC0010003' and includes a search bar and navigation tabs (Discuss, Follow, Update, Resolve, Delete, Responses). The form fields are organized into two columns. The left column contains fields for Number (INC0010003), Caller (Abel Tuter), Category (Inquiry / Help), Subcategory (None), Service, Service offering, Configuration item, Short description (hil), and Description. The right column contains fields for Channel (None), State (New), Impact (2 - Medium), Urgency (1 - High), Priority (2 - High), Assignment group (Software), and Assigned to. Below the form, there are tabs for Notes, Related Records, and Resolution Information. The Notes tab is active, showing a 'Watch list' and a 'Work notes list'. The 'Work notes list' contains a single entry 'Work notes' with a 'Post' button. At the bottom right, there is a checkbox for 'Comments (Customer visible)' and a 'Post' button.

7.4 Activity 4 – State Change Through List Editing

The Incident list was opened through **Incident** → **All**. The **State** field was double-clicked to perform a direct list edit.

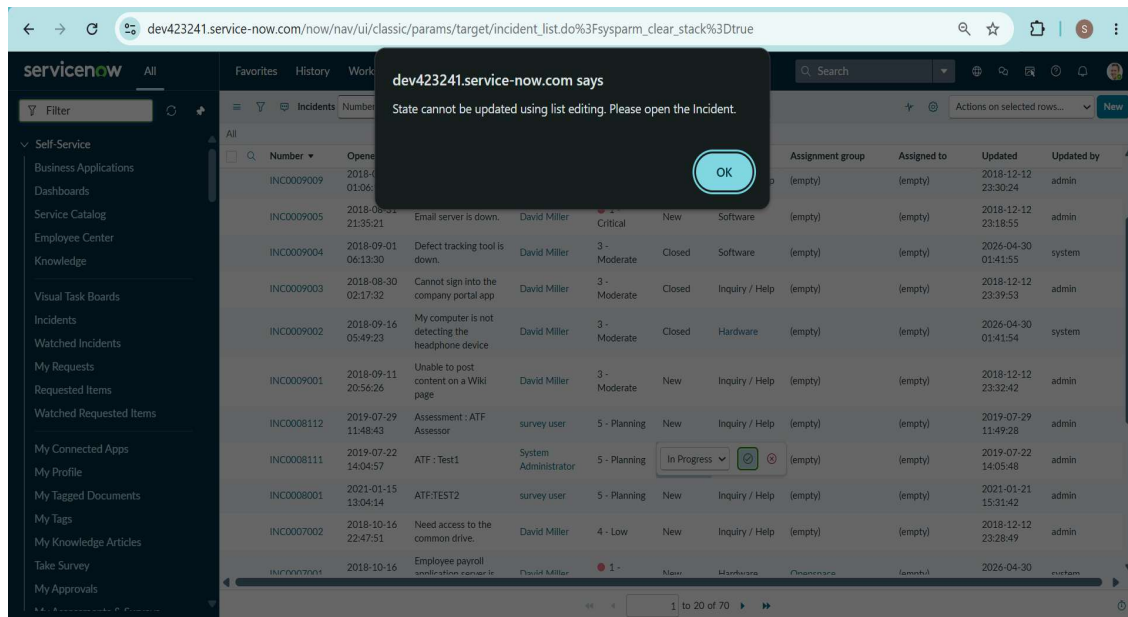
Expected Result:

Direct State changes through list editing should be blocked and an alert message should be displayed.

Actual Result:

The alert message was displayed and the State value remained unchanged.

Screenshot:



7.5 Activity 5 – State Change Through Incident Form

The same Incident was opened from the Incident form. The **State** value was changed and the record was updated.

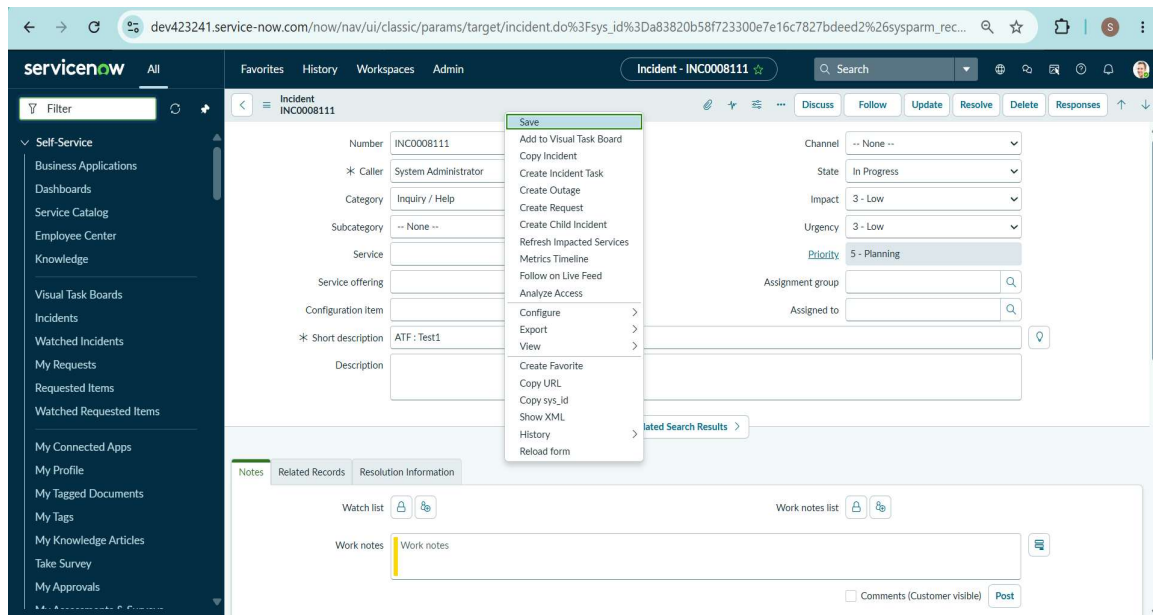
Expected Result:

State changes made through the Incident form should be saved successfully.

Actual Result:

The State change was saved successfully through the Incident form, while the other configured rules continued to work.

Screenshot:



Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0009002	2018-09-16 05:49:23	my computer is not detecting the headphone device	David Miller	3 - Moderate	Closed	Hardware	(empty)	(empty)	2026-04-30 01:41:54	system
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	4 - Low	In Progress	Inquiry / Help	(empty)	(empty)	2026-09-21 21:32:15	admin
INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
INC0007001	2018-10-16 22:47:10	Employee payroll application server is down.	David Miller	1 - Critical	New	Hardware	Openspace	(empty)	2026-04-30 01:32:04	system
INC0001990	2020-06-07 09:02:25	Unable to access the personal details section in payroll portal	Problem CoordinatorATF	5 - Planning	On Hold	Inquiry / Help	(empty)	(empty)	2020-06-07 09:04:34	admin
INC0000601	2026-04-25 02:42:59	The USB port on my PC stopped working	Beth Anglin	4 - Low	Closed	Hardware	(empty)	(empty)	2026-09-20 09:30:54	system
INC0000080	2016-12-12 07:19:57	Unable to connect to email	Joe Employee	3 - Moderate	Closed	Inquiry / Help	Network	David Loo	2016-12-13 18:46:44	employee

7.6 Testing Summary

The project was tested using different Incident scenarios to verify the configured **UI Policy**, **UI Policy Action**, **onChange Client Script**, **onSubmit Client Script**, and **onCellEdit Client Script**. The testing confirmed that the configured field behavior, validation, automatic value update, and State editing restriction worked as intended.

8. PROJECT DOCUMENTATION PHASE

8.1 Project Documentation

The project documentation was prepared to record the complete implementation and testing process of “**Implement Client Script & UI Policy (Incident)**” in ServiceNow.

The documentation includes the project requirements, design, planning, development activities, testing activities, screenshots, and final results.

8.2 Documentation Contents

The following activities were documented:

- Project problem identification
- Project objective and proposed solution
- Functional and software requirements
- UI Policy design
- UI Policy Action configuration
- Client Script configuration
- onChange Client Script implementation
- onSubmit Client Script implementation
- onCellEdit Client Script implementation
- Functional testing and verification
- Screenshots of important configurations and results
- Final project outcome

8.3 Screenshots and Evidence

Screenshots were captured during the project implementation and testing to provide evidence of the completed tasks.

The main screenshots include:

1. **High Impact Control – UI Policy**
2. **Urgency – UI Policy Action**
3. **onChange Client Script**
4. **Automatic Urgency Testing**
5. **onSubmit Client Script**
6. **Prevent Save Testing**
7. **onCellEdit Client Script**
8. **State List Edit Restriction Testing**
9. **Reverse Condition Testing**

10. State Form Update Testing

8.4 Final Documentation

All the implementation steps and testing results were organized into the project documentation. The screenshots were placed alongside the relevant tasks to clearly show the configuration and working results.

The completed documentation provides a record of the project from **planning to implementation and testing**.

8.5 Documentation Outcome

The documentation successfully records the implementation of UI Policies and Client Scripts for the Incident table and provides evidence of the testing activities performed during the project.

9. PROJECT DEMONSTRATION PHASE

9.1 Project Demonstration

The completed “**Implement Client Script & UI Policy (Incident)**” project was demonstrated in the ServiceNow environment. The demonstration explains the purpose of the project, the configured UI Policies and Client Scripts, their working process, and the final testing results.

9.2 Demonstration Content

The project demonstration includes the following:

- **Project Name:** Implement Client Script & UI Policy (Incident)
- **Purpose of the Project:** To improve data accuracy and control Incident form behavior.
- **UI Policy:** High Impact Control
- **UI Policy Action:** Make Urgency read-only for High Impact Incidents.
- **onChange Client Script:** Automatically set Urgency to High.
- **onSubmit Client Script:** Prevent saving when Assigned To is missing.
- **onCellEdit Client Script:** Prevent State changes through list editing.
- **Testing:** Verification of all configured rules using different Incident scenarios.

- **Final Output:** Successful implementation and testing of the required Incident controls.

9.3 Demonstration Process

The demonstration follows the project workflow:

Incident Form → Impact = High → UI Policy & Client Scripts → Field Control → Validation → Testing → Final Result

The screen recording shows the actual ServiceNow configuration and execution of the project. A voice-over is used to explain the purpose, implementation steps, working process, and results.

9.4 Final Demonstration Result

The demonstration shows that the configured UI Policies and Client Scripts perform the required Incident field controls and validations. The completed project was tested using the specified scenarios and the results were documented with screenshots.

10. CONCLUSION

The “**Implement Client Script & UI Policy (Incident)**” project demonstrates how ServiceNow UI Policies and Client Scripts can be used to control Incident records. The project implements dynamic field behavior, automatic value updates, validation during submission, and restrictions on direct list editing.

The project was developed and tested through different Incident scenarios. The testing confirmed the configured functionality and provided a practical demonstration of using client-side controls in ServiceNow Incident Management.

REFERENCES

1. **ServiceNow Developer Platform** – ServiceNow platform and Personal Developer Instance environment used for project implementation.
2. **ServiceNow Incident Management** – Used for creating, updating, and testing Incident records.
3. **ServiceNow UI Policies** – Used to control Incident form fields based on

conditions.

4. **ServiceNow UI Policy Actions** – Used to configure field behavior such as read-only and mandatory settings.
5. **ServiceNow Client Scripts** – Used for `onChange`, `onSubmit`, and `onCellEdit` client-side behavior.
6. **SmartBridge Project Document – “Implement Client Script & UI Policy (Incident)”** – Used as the primary project guide for requirements, tasks, implementation, and testing.